

```
# 1 Create a Student class with a constructor that initializes name and age. Display the student's details.
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Student Age: {self.age}")
s=Student("Alice", 20)
s.display_details()
```

Student Name: Alice
Student Age: 20

```
# 2 Create a Car class with a constructor that initializes brand and price. Print the values.
class Car:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

    def display_details(self):
        print(f"Car Brand: {self.brand}")
        print(f"Car Price: ${self.price}")

c = Car("Toyota", 25000)
c.display_details()
```

Car Brand: Toyota
Car Price: \$25000

```
# 3 Create a Rectangle class with a constructor that accepts length and width. Calculate and display the area.
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def calculate_area(self):
        area = self.length * self.width
        print(f"Area of Rectangle: {area}")
r=Rectangle(5, 3)
r.calculate_area()
```

Area of Rectangle: 15

```
# 4 Create a Person class with a constructor that initializes name and gender. Display the information.
class Person:
    def __init__(self, name, gender):
        self.name = name
        self.gender = gender

    def display_info(self):
        print(f"Person Name: {self.name}")
        print(f"Person Gender: {self.gender}")
p=Person("Bob", "Male")
p.display_info()
```

Person Name: Bob
Person Gender: Male

```
# 5 Create a Book class with a constructor that accepts title and author. Display the book details.
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def display_details(self):
        print(f"Book Title: {self.title}")
        print(f"Book Author: {self.author}")
b=Book("The Great", "F. Scott")
b.display_details()
```

Book Title: The Great
Book Author: F. Scott

```
# 6 Create an Employee class with a constructor that initializes id, name, and salary. Display all details.
class Employee:
    def __init__(self, id, name, salary):
        self.id = id
```

```
self.name = name
self.salary = salary

def display_details(self):
    print(f"Employee ID: {self.id}")
    print(f"Employee Name: {self.name}")
    print(f"Employee Salary: ${self.salary}")

e = Employee(1, "Alice", 50000)
e.display_details()
```

Employee ID: 1
Employee Name: Alice
Employee Salary: \$50000

```
# 7 Create a BankAccount class with a constructor that initializes accountNumber and balance. Display the account info
class BankAccount:
    def __init__(self, accountNumber, balance):
        self.accountNumber = accountNumber
        self.balance = balance

    def display_account_info(self):
        print(f"Account Number: {self.accountNumber}")
        print(f"Account Balance: ${self.balance}")
b=BankAccount("123456789", 1000)
b.display_account_info()
```

Account Number: 123456789
Account Balance: \$1000

```
# 8 Create a Mobile class with a constructor that accepts brand, model, and price. Print the details.
class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def display_details(self):
        print(f"Mobile Brand: {self.brand}")
        print(f"Mobile Model: {self.model}")
        print(f"Mobile Price: ${self.price}")
m=Mobile("Samsung", "Galaxy S21", 799)
m.display_details()
```

Mobile Brand: Samsung
Mobile Model: Galaxy S21
Mobile Price: \$799

```
# 9 Create a Circle class with a constructor that accepts radius. Calculate and display the area of the circle.
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def calculate_area(self):
        area = 3.14 * (self.radius ** 2)
        print(f"Area of Circle: {area}")
c=Circle(4)
c.calculate_area()
```

Area of Circle: 50.24

```
# 10 Create a Product class with a constructor that initializes productName and quantity. Display the product details.
class Product:
    def __init__(self, productName, quantity):
        self.productName = productName
        self.quantity = quantity

    def display_details(self):
        print(f"Product Name: {self.productName}")
        print(f"Product Quantity: {self.quantity}")
p=Product("Laptop", 10)
p.display_details()
```

Product Name: Laptop
Product Quantity: 10

```
# 11 Write a program demonstrating a default constructor. Create a Student class whose constructor prints "Student obj
class Student:
    def __init__(self):
        print("Student object created")
s=Student()
```

Student object created

```
# 12 Write a program demonstrating a parameterized constructor. Create a Laptop class with brand and ram as attributes
class Laptop:
    def __init__(self, brand, ram):
        self.brand = brand
        self.ram = ram

    def display_details(self):
        print(f"Laptop Brand: {self.brand}")
        print(f"Laptop RAM: {self.ram}GB")
l=Laptop("Dell", 16)
l.display_details()
```

Laptop Brand: Dell
Laptop RAM: 16GB

```
# 13 Create a Box class with a constructor that accepts length, width, and height. Calculate and display the volume.
class Box:
    def __init__(self, length, width, height):
        self.length = length
        self.width = width
        self.height = height

    def calculate_volume(self):
        volume = self.length * self.width * self.height
        print(f"Volume of Box: {volume}")
b=Box(4, 5, 6)
b.calculate_volume()
```

Volume of Box: 120

```
# 14 Create a Movie class with a constructor that accepts title, director, and rating. Create an object and display al
class Movie:
    def __init__(self, title, director, rating):
        self.title = title
        self.director = director
        self.rating = rating

    def display_details(self):
        print(f"Movie Title: {self.title}")
        print(f"Movie Director: {self.director}")
        print(f"Movie Rating: {self.rating}")
m=Movie("Inception", "Christopher Nolan", 8.8)
m.display_details()
```

Movie Title: Inception
Movie Director: Christopher Nolan
Movie Rating: 8.8

```
# 15 Create a class MobilePhone with attributes brand and storage. Use a constructor to initialize them. Create a meth
class MobilePhone:
    def __init__(self, brand, storage):
        self.brand = brand
        self.storage = storage

    def display(self):
        print(f"Mobile Phone Brand: {self.brand}")
        print(f"Mobile Phone Storage: {self.storage}GB")
m=MobilePhone("Apple", 128)
m.display()
```

Mobile Phone Brand: Apple
Mobile Phone Storage: 128GB

```
# 16 Create a class Teacher with attributes name and subject. Use a constructor to initialize them. Add a method teach
class Teacher:
    def __init__(self, name, subject):
        self.name = name
        self.subject = subject

    def teach(self):
        print(f"Teacher {self.name} is teaching {self.subject}")
t=Teacher("Mr. Smith", "Mathematics")
t.teach()
```

Teacher Mr. Smith is teaching Mathematics

```
# 17 Create a class Product with attributes name and price. Use a constructor to initialize them. Add a method discount
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def discount(self):
        discounted_price = self.price * 0.9
        print(f"Product Name: {self.name}")
        print(f"Original Price: ${self.price}")
        print(f"Final Price: ${self.price-discounted_price} with a discount ${discounted_price}")
p=Product("Smartphone", 500)
p.discount()
```

Product Name: Smartphone
Original Price: \$500
Final Price: \$50.0 with a discount \$450.0

```
# 18 Create a class House with attributes address and rooms. Use a constructor to initialize them. Add a method display
class House:
    def __init__(self, address, rooms):
        self.address = address
        self.rooms = rooms

    def display(self):
        print(f"House Address: {self.address}")
        print(f"Number of Rooms: {self.rooms}")
h=House("123 Main St", 4)
h.display()
```

House Address: 123 Main St
Number of Rooms: 4

```
# 19 Create a class Animal with attributes name and sound. Use a constructor to initialize them. Add a method make_sound
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def make_sound(self):
        print(f"{self.name} makes a sound: {self.sound}")
a=Animal("Dog", "Bark")
a.make_sound()
```

Dog makes a sound: Bark

```
# 20 Create a class Bike with attributes brand and speed. Use a constructor to initialize them. Add a method accelerate
class Bike:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed

    def accelerate(self):
        self.speed += 10
        print(f"Bike Brand: {self.brand}")
        print(f"New Speed: {self.speed} km/h")
b=Bike("Yamaha", 50)
b.accelerate()
```

Bike Brand: Yamaha
New Speed: 60 km/h

```
# 21 Create a class Movie with attributes title and rating. Use a constructor to initialize them. Add a method display
class Movie:
    def __init__(self, title, rating):
        self.title = title
        self.rating = rating
```

```
def display_rating(self):
    print(f"Movie Title: {self.title}")
    print(f"Rating: {self.rating}")
m=Movie("Inception", 8.8)
m.display_rating()
```

Movie Title: Inception
Rating: 8.8

```
# 22 Create a class ShoppingCart with attributes item and price. Use a constructor to initialize them. Add a method to
class ShoppingCart:
    def __init__(self, item, price):
        self.item = item
        self.price = price

    def total(self):
        print(f"Item: {self.item}")
        print(f"Price: ${self.price}")
s=ShoppingCart("Laptop",1000)
s.total()
```

Item: Laptop
Price: \$1000

```
# 23 Create a class Player with attributes name and score. Use a constructor to initialize them. Add a method add_score
class Player:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def add_score(self):
        self.score += 10
        print(f"Player Name: {self.name}")
        print(f"Updated Score: {self.score}")
p=Player("John", 50)
p.add_score()
```

Player Name: John
Updated Score: 60

```
# 24 Create a class SmartWatch with attributes brand and battery. Use a constructor to initialize them. Add a method charge
class SmartWatch:
    def __init__(self, brand, battery):
        self.brand = brand
        self.battery = battery

    def charge(self):
        self.battery += 20
        print(f"SmartWatch Brand: {self.brand}")
        print(f"New Battery Level: {self.battery}%")
s=SmartWatch("Fitbit", 50)
s.charge()
```

SmartWatch Brand: Fitbit
New Battery Level: 70%